

C++ for Quantitative Finance

1 Foundations of C++

1.1 Why C++ in Quantitative Finance

C++ gives the programmer direct control over memory and execution, which is why pricing libraries, risk engines and low latency trading systems are commonly written in it.

- Much of the syntax of modern languages (Java, C#, JavaScript) descends from C and C++
- The interpreters and numerical cores of Python, R and Matlab are themselves written in C or C++
- The cost of this control is complexity: the language will let you make mistakes that higher level languages prevent

Toolchain: A C++ project needs a compiler (e.g. GCC, Clang or MSVC; each may translate the same code slightly differently), an editor or IDE, and usually a build system such as CMake to manage compilation across platforms. Fixing a language standard (e.g. C++20) keeps behaviour consistent.

Good Practice: Work in small increments, compiling and testing after each change. Plan difficult algorithms on paper before coding. When something fails, reduce the problem to its smallest failing case instead of patching blindly. Reading documentation efficiently is a core skill.

1.2 A Short History

C was created at Bell Labs in the early 1970s alongside UNIX, with the aim of writing portable system software. It was standardised by ANSI in 1989 and ISO in 1990, with later revisions C99, C11, C17/18 and C23.

C++ began in 1979 as “C with Classes” by Bjarne Stroustrup, adding object orientation and generic programming. ISO revisions: C++98, 03, 11, 14, 17, 20, 23.

- Backwards compatibility is a priority, so old code (and most C code) still compiles
- Features are rarely removed, which makes the language large
- Compiler support for new features arrives gradually
- Migrating a large codebase to a new standard is often too costly to justify

1.3 From Source Code to Executable

Source files (.cpp) hold the program logic and the entry point **main**. **Header files** (.h/.hpp) hold declarations that several source files need to share.

Build Stages:

1. **Preprocessing:** directives such as **#include** are expanded into each source file
2. **Compilation:** each processed source file becomes an object (binary) file
3. **Linking:** object files and external libraries are joined, resolving references between them, into one executable

```
#include <iostream>

int main()
{
    std::cout << "Pricing engine started\n";
    return 0;    // zero signals normal termination
}
```

Comments: *// single line* and */* block */*. Comments are discarded by the compiler; use them to explain intent or non obvious steps rather than restating the code.

Output: `std::cout` writes to the terminal and values can be chained, e.g. `std::cout << "spot = " << 101.5 << "\n";`

2 Types, Variables and Operators

2.1 Built In Types

The exact size of each type depends on the platform. Query the limits with `std::numeric_limits<T>` from `<limits>` (members `min()`, `max()`, `lowest()`).

Type	Typical range	Bytes
char	−128 to 127	1
int	about $\pm 2.1 \times 10^9$	4
float	$\pm 3.4 \times 10^{\pm 38}$, about 7 significant digits	4
double	$\pm 1.7 \times 10^{\pm 308}$, about 15 significant digits	8

Modifiers: **short** and **long** shrink or widen integer ranges. **unsigned** forbids negative values and roughly doubles the positive range for the same storage; **signed** is the default. In pricing code, prefer **double** for prices and rates.

sizeof: An operator (evaluated by the compiler, not a library function) returning the size in bytes of a type or expression, e.g. `sizeof(double)` is typically 8.

2.2 Variables

C++ is **statically typed**: every variable has a fixed type chosen at declaration, and it must be declared before use.

```
|double spot;                // declared, value undefined
|double strike = 100.0;      // declared and initialised
|int n_steps = 250, n_paths = 10000; // several on one line
```

Always initialise variables; uninitialised locals hold arbitrary values.

2.3 Operators

Arithmetic (+ − ∗ / %): the result type follows the operand types. With integers, `7 / 2` is 3 and `7 % 2` is 1; with doubles, `7.0 / 2.0` is 3.5. A frequent bug is `1 / 252` evaluating to 0; write `1.0 / 252`.

Comparison (== != < > <= >=) yields a `bool`. Any nonzero value converts to **true** and zero to **false**. Avoid == between doubles; compare against a tolerance instead.

Logical: **!** negates, **&&** is true only if both sides are true, **||** is true if at least one side is true. Both **&&** and **||** stop evaluating as soon as the result is known.

Compound Assignment: `x += y` is shorthand for `x = x + y`; likewise `-- *= /= %=`.

Increment:

```
|int k = 5;
|int a = ++k;    // k becomes 6 first, then a = 6
|int b = k++;    // b = 6, then k becomes 7
```

Ternary Operator: `condition ? value_if_true : value_if_false`
`|double payoff = (spot > strike) ? spot - strike : 0.0;`

Conversions: Widening (e.g. `int` to `double`) happens silently. Narrowing loses information and should be written explicitly with **static_cast**:

```
|double days = 91.7;
|int whole_days = static_cast<int>(days);    // 91, truncated
```

Precedence: Standard mathematical order applies (multiplication before addition, and so on). Parentheses override it; use them where they make intent clearer, but not excessively.

3 Control Flow and Functions

3.1 Blocks and Scope

A block is a sequence of statements between braces { }. Variables declared inside a block live on the **stack** and are destroyed automatically when execution leaves the block. Indent the contents of each block consistently.

3.2 Branching

if / else selects between paths using any Boolean expression. **switch** compares one integral value against a list of compile time constants.

```
|if (spot >= barrier)
|{
|    knocked_out = true;
|}
|else
|{
|    payoff = std::max(spot - strike, 0.0);
|}

|switch (option_type)    // e.g. 0 = call, 1 = put
|{
|    case 0:
|        value = call_price(spot, strike);
|        break;    // without break, execution falls through
|    case 1:
|        value = put_price(spot, strike);
|        break;
|    default:
|        value = 0.0;
|}
```

3.3 Loops

```
|// for: known number of iterations
|double pv = 0.0;
|for (int t = 1; t <= n_years; ++t)
|    pv += coupon / std::pow(1.0 + y, t);

|// while: condition checked before each pass
|while (error > tolerance) { /* refine estimate */ }

|// do while: body always runs at least once
|do { /* draw a sample */ } while (!accepted);
```

break exits the innermost loop immediately; **continue** skips the rest of the current iteration.

3.4 Functions

A function is identified by its **return type**, **name** and **parameter list**. A **declaration** (prototype) states the signature and usually lives in a header; the **definition** supplies the body.

```
|double discount_factor(double r, double t); // declaration

|double discount_factor(double r, double t) // definition
|{
|    return std::exp(-r * t);
|}
```

- A function with no result has return type `void`
- Every call pushes a **stack frame** holding arguments, locals and the return address, so calls cost more than simple operators

3.5 Recursion

A recursive function calls itself on a smaller problem until it reaches a base case. In **tail recursion** the recursive call is the very last operation, so the compiler can reuse the current stack frame and the recursion runs like a loop. Passing an accumulator is the usual way to achieve this.

```
|// plain recursion: x^n for integer n >= 0
|double power(double x, unsigned int n)
|{
|    if (n == 0)
|        return 1.0;
|    return x * power(x, n - 1); // work remains after the call
|}

|// tail recursive version with accumulator
|double power_acc(double x, unsigned int n, double acc)
|{
|    if (n == 0)
|        return acc;
```

```
    return power_acc(x, n - 1, acc * x);    // call is last
}
```

4 The Preprocessor and Scope

4.1 Preprocessor Directives

Lines beginning with **#** are handled by the preprocessor, a text substitution pass that runs before compilation.

Includes: `#include` <vector> searches the system paths; `#include` "utils/pricing.hpp" searches relative to the project first.

Macros: `#define` performs literal text replacement, with no type checking.

```
#define DAYS_PER_YEAR 365
#define cube(x) x * x * x
cube(3);           // 27
cube(1 + 1);       // 1 + 1 * 1 + 1 * 1 + 1 = 4, not 8
```

Prefer `constexpr` constants and inline functions, which are type safe.

4.2 Conditional Compilation and Include Guards

`#if`, `#ifdef`, `#ifndef`, `#elif`, `#else` and `#endif` include or exclude code before compilation. Their main use is the **include guard**: when two headers both include a common header, a source file using both would otherwise see the same declarations twice.

```
#ifndef PRICING_UTILS_HPP
#define PRICING_UTILS_HPP
double discount_factor(double r, double t);
#endif
```

The single line `#pragma once` achieves the same effect; it is non standard but supported by all major compilers.

4.3 Scope and Lifetime

- Block scope:** visible only inside its braces, destroyed at the closing brace

- File scope:** declared outside any function, visible within that file

- Global scope:** visible throughout the program

Internal Linkage: Marking a file scope variable `static` or `const` restricts it to its own translation unit (source file). `const` additionally makes it read only, so shared constants can safely live in headers, whereas `static` variables belong in source files. Global mutable state should generally be avoided.

Static Locals: A `static` variable inside a function is initialised once and keeps its value between calls; its scope is unchanged but its lifetime extends to the whole program.

```
int next_trade_id()
{
    static int counter = 0;
    return ++counter;
}
// successive calls return 1, 2, 3, ...
```

5 Pointers, Arrays and Memory

5.1 Pointers

A pointer holds the memory address of another object. `T* p` declares a pointer to `T`, `&x` takes the address of `x`, and `*p` **dereferences** `p` to reach the object.

```
double notional = 1e6;
double* p = &notional;
*p *= 2.0;           // notional is now 2e6

void apply_haircut(double* value, double h)
{
    *value *= (1.0 - h);    // modifies the caller's variable
}
apply_haircut(&notional, 0.05);
```

Passing a pointer copies only the address, so the function can change data that lives outside its own scope.

5.2 Arrays

An array stores elements of one type in **consecutive** memory locations, indexed from zero. Element *i* sits at the start address plus *i*× `sizeof(T)`.

```
double rates_c[4] = {};           // C style
std::array<double, 4> rates = {0.01, 0.015, 0.02, 0.025};    // <array>
double* first = rates.data();     // raw pointer to element 0
```

`std::array` knows its own size, supports bounds checked access through `at()`, and can be copied, unlike a C style array, which **decays** to a pointer when passed to a function or used to initialise a pointer.

Pointer Arithmetic: Adding *k* to a pointer moves it *k* elements, not *k* bytes.

```
double* q = rates.data();
q += 3;           // now at rates[3]
--q;             // now at rates[2]
double r2 = *q;  // 0.02
```

5.3 Stack and Heap

The stack is small (typically a few megabytes) and its sizes are fixed at compile time. Large or runtime sized data belongs on the **heap**, requested with `new` and returned with `delete`.

```
std::size_t n_paths = 100000;
double* terminal = new double[n_paths]();    // zero initialised
// ... simulate ...
delete[] terminal;    // array form must match new[]
```

Memory Leak: If a function allocates with `new` and returns without `delete`, that memory stays reserved but unreachable. Repeated calls (e.g. once per simulation) can exhaust available memory.

5.4 std::vector

`std::vector<T>` (from <vector>) is a resizable heap array that frees its memory automatically when it goes out of scope.

```
std::vector<double> path(n_steps + 1, 0.0);
path[0] = spot;
std::vector<double> prices;
prices.reserve(1000);    // capacity only, size still 0
prices.push_back(101.2);    // size 1
```

- `size()` returns the number of elements

- `resize(n)` changes the number of elements

- `reserve(n)` preallocates capacity without changing the size

- `push_back(v)` appends, reallocating if capacity is exhausted

6 Option Pricing with Binomial Trees

6.1 Pricing Framework

Under no arbitrage, the value of a derivative equals its expected payoff under a **risk neutral measure**, discounted at the risk free rate. Closed forms exist only in special cases, so in practice we rely on:

- Lattice methods:** binomial or trinomial trees that discretise time and price

- Numerical integration:** trapezium rule or Gaussian quadrature over the payoff density

- Monte Carlo:** averaging simulated payoffs, well suited to many risk factors

6.2 European Options and Black Scholes

A European call (put) grants the right, without obligation, to buy (sell) the underlying for strike *K* at maturity *T*. With spot *S*₀, risk free rate *r*, continuous dividend yield *q* and volatility *σ*:

$$C_0 = S_0 e^{-qT} N(d_1) - K e^{-rT} N(d_2) \qquad P_0 = K e^{-rT} N(-d_2) - S_0 e^{-qT} N(-d_1)$$

$$d_1 = \frac{\ln(S_0/K) + (r - q + \frac{1}{2}\sigma^2) T}{\sigma\sqrt{T}}, \qquad d_2 = d_1 - \sigma\sqrt{T}$$

$$N(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-y^2/2} dy$$

In C++, $N(x) = \frac{1}{2} \operatorname{erfc}(-x/\sqrt{2})$ using `std::erfc` from <cmath>.

Assumptions: No transaction costs, the underlying follows geometric Brownian motion (log normal prices), and volatility and rates are constant to maturity. The closed form is the natural benchmark for validating a numerical method.

American Options: Early exercise removes the closed form (except in special cases such as perpetual options), so a numerical method is required.

6.3 The CRR Binomial Model

Divide $[0,T]$ into *n* steps of length $\Delta t = T/n$. Over each step the price moves to *uS* or *dS*, and the Cox Ross Rubinstein choice makes the tree **recombine** (an up move followed by a down move returns to the same node), so after *n* steps there are only *n* + 1 terminal nodes instead of *2ⁿ*.

$$u = e^{\sigma\sqrt{\Delta t}}, \qquad d = \frac{1}{u}, \qquad p = \frac{e^{(r-q)\Delta t} - d}{u - d}$$

Deriving the Parameters: The size of the jump is fixed by volatility, the probability by risk neutrality.

- Variance matching:** setting the one step variance of the log return, $p(\ln u)^2 + (1-p)(\ln d)^2 - [p\ln u + (1-p)\ln d]^2$, equal to $\sigma^2\Delta t$ with $\ln d = -\ln u$ gives $u = e^{\sigma\sqrt{\Delta t}}$ (exactly for $p = \frac{1}{2}$, approximately otherwise)

- Risk neutrality:** requiring $pu + (1-p)d = e^{(r-q)\Delta t}$ gives the expression for *p* above

Valid Probabilities: $0 < p < 1$ holds if and only if

$$d < e^{(r-q)\Delta t} < u \quad \Longleftrightarrow \quad \Delta t < \frac{\sigma^2}{(r-q)^2}$$

Convergence: As $n \rightarrow \infty$ the terminal distribution approaches the log normal and European prices converge to Black Scholes. Convergence is oscillatory rather than monotone, because the position of the strike relative to the terminal nodes shifts as *n* changes.

6.4 Backward Induction

Let *V_i^j* be the option value at step *i* after *j* up moves, with node price *S_i^j* = *S*₀*u^jd^{i-j}*. Terminal values are the payoffs, and earlier values follow from

$$V_i^j = e^{-r\Delta t} \left[p V_{i+1}^{j+1} + (1-p) V_{i+1}^j \right]$$

For an American option, compare with immediate exercise at every node: *V_i^j* = max (payoff(*S_i^j*), *e^{-rΔt}*[...]).

Two Implementations:

- Recursive:** evaluate *V*₀⁰ by calling the recurrence on its two children until terminal nodes are reached. Simple to write, but without memoisation shared nodes are recomputed and the cost grows like *2ⁿ*

- Iterative:** fill a vector with the *n* + 1 payoffs and roll it back one step at a time, overwriting in place; the vector shrinks by one element per step and cost is *O*(*n*²)

```
double crr_price(double s0, double k, double r, double q, double sigma, double t, int
n, bool is_call, bool is_american)
{
    double dt = t / n;
    double u = std::exp(sigma * std::sqrt(dt));
    double d = 1.0 / u;
    double p = (std::exp((r - q) * dt) - d) / (u - d);
    double disc = std::exp(-r * dt);
    std::vector<double> v(n + 1);
    for (int j = 0; j <= n; ++j)
    {
        double s = s0 * std::pow(u, j) * std::pow(d, n - j);
        v[j] = is_call ? std::max(s - k, 0.0) : std::max(k - s, 0.0);
    }
    for (int i = n - 1; i >= 0; --i)
    {
        for (int j = 0; j <= i; ++j)
```

```

    {
        v[j] = disc * (p * v[j + 1] + (1.0 - p) * v[j]);
        if (is_american)
        {
            double s = s0 * std::pow(u, j) * std::pow(d, i - j);
            double ex = is_call ? s - k : k - s;
            v[j] = std::max(v[j], ex);
        }
    }
}
return v[0];
}
```

Validation: First confirm the code implements the algorithm correctly (small trees checked by hand, put call parity for European prices, American value never below European value). Then study accuracy, e.g. error against Black Scholes as n grows.

7 Data Structures and Classes

7.1 Abstract Data Types and Data Structures

An **abstract data type (ADT)** specifies *what* a container does (its operations and their behaviour); a **data structure** is a concrete implementation specifying *how*. The standard library supplies `std::vector<T>`, `std::list<T>`, `std::map<K, V>` and `std::unordered_map<K, V>`; other languages implement the same ADTs differently.

Operation	Array	Linked list	Tree	Hash
Lookup	$O(1)$	$O(n)$	$O(\log n)$	$O(1)$ avg
Insert in middle	$O(n)$	$O(1)^*$	$O(\log n)$	$O(1)$ avg
Insert at front	$O(n)$	$O(1)$	$O(\log n)$	n/a
Insert at back	$O(1)$ amort.	$O(1)$	$O(\log n)$	n/a

* Once the position is known; finding it costs $O(n)$. Lookup is by position for arrays and lists, by key for trees and hash tables.

- **Dynamic array (vector):** contiguous, cache friendly, fastest to iterate
- **Doubly linked list (list):** each node points to its neighbours
- **Balanced tree (map):** keys kept sorted, e.g. a yield curve keyed by maturity
- **Hash table (unordered_map):** a hash function sends each key to a bucket index; fast lookup, e.g. prices keyed by ticker, but unordered and slower to iterate than an array

7.2 Classes and Objects

A **class** is a user defined type bundling data (**members**) with the functions that act on it (**member functions**). An **object** is one instance of a class; `std::vector<double> v`; creates an object `v` of class `std::vector<double>`.

Access: Members of a class are **private** unless stated; members of a **struct** are **public** unless stated. Otherwise the two keywords are equivalent in C++.

Lifecycle: The **constructor** runs when an object is created and the **destructor** (`~Name()`) when it is destroyed. If none are written, the compiler generates defaults.

```
// bond.hpp
#pragma once
class Bond
{
private:
    double m_face;
    double m_coupon_rate;
    int m_years;
public:
    Bond(double face, double coupon_rate, int years);
    double price(double yield) const;
};

// bond.cpp
#include "bond.hpp"
#include <cmath>
Bond::Bond(double face, double coupon_rate, int years)
    : m_face(face), m_coupon_rate(coupon_rate), m_years(years)
{}
```

```
double Bond::price(double yield) const
{
    double c = m_face * m_coupon_rate;
    double pv = 0.0;
    for (int t = 1; t <= m_years; ++t)
        pv += c / std::pow(1.0 + yield, t);
    return pv + m_face / std::pow(1.0 + yield, m_years);
}

// main.cpp
Bond b(100.0, 0.05, 3);
double p = b.price(0.05);    // 100 at par
```

The **member initialiser list** (after the colon) constructs members directly. It gives the same result as assigning in the body for simple types, but is required for **const** and reference members and avoids a default construction followed by assignment.

8 Object Oriented Design

8.1 The Four Pillars

Object orientation organises a program around objects that combine state and behaviour. For large systems this makes code easier to reason about, to test in isolation and to extend. As a running example, take a base class **Instrument** with a virtual `price()` method, specialised by **Bond** and **EuropeanOption**.

Abstraction: A risk system only needs to ask an instrument for its price; how that price is computed is irrelevant to the caller. Expose a small, clear interface and hide the complexity behind it. Abstract base classes express such an interface without committing to an implementation.

Encapsulation: Each object owns its data and is the only code allowed to change it. Callers interact through public member functions and treat the object as a black box. Aim for high cohesion inside a class and low coupling between classes.

Inheritance: **Bond** and **EuropeanOption** derive from **Instrument**, reusing its common parts and adding their own. A derived object must be usable wherever the base type is expected without surprising the caller (the Liskov substitution principle).

Polymorphism: Each derived class overrides `price()` with its own formula. A call through a base pointer or reference dispatches at runtime to the correct version, so a collection of mixed instruments is handled uniformly.

```
class Instrument
{
public:
    virtual double price() const = 0;    // pure virtual
    virtual ~Instrument() = default;    // virtual destructor
};

class EuropeanOption : public Instrument
{
private:
    double m_spot, m_strike, m_rate, m_vol, m_expiry;
public:
    EuropeanOption(double s, double k, double r, double v, double t)
        : m_spot(s), m_strike(k), m_rate(r), m_vol(v), m_expiry(t) {}
    double price() const override { /* Black Scholes */ return 0.0; }
};

std::vector<Instrument*> book = {&bond_a, &call_b, &put_c};
double total = 0.0;
for (const Instrument* inst : book)
    total += inst->price();    // correct override chosen at runtime
```

A base class used polymorphically needs a **virtual destructor**, otherwise deleting a derived object through a base pointer skips the derived destructor.

8.2 Organising a Class Header

A consistent layout makes classes easier to read. One common ordering:

1. Data members
2. Constructors and destructor
3. Accessors: **const** functions that only read state
4. Mutators: functions that change state

9 Functions, Arguments and const

9.1 Overloading

Functions (including member functions) may share a name provided their **parameter lists** differ in number or type. The compiler picks the best match for the arguments at each call.

```
double year_fraction(int days) { return days / 365.0; }
double year_fraction(int days, int basis) { return static_cast<double>(days) / basis; }

year_fraction(182);           // 0.4986
year_fraction(180, 360);      // 0.5
```

What does not count as a new overload:

- Changing only the **return type**
- Changing only the **parameter names** or the body (this is a redefinition)
- Pairs such as `f(int)` and `f(int&)` can be declared, but a call like `f(x)` is ambiguous and fails to compile

By contrast, `f(int)` and `f(int*)` are distinct overloads.

9.2 Passing Arguments

```
void bump_value(double v) { v += 0.0001; }    // copy
void bump_ptr(double* v) { *v += 0.0001; }    // address
void bump_ref(double& v) { v += 0.0001; }    // alias

double rate = 0.03;
bump_value(rate);    // rate unchanged
bump_ptr(&rate);     // rate = 0.0301
bump_ref(rate);      // rate = 0.0302
```

Pointers vs References:

	Pointer	Reference
Can be null	Yes	No
Can be reseated	Yes	No, bound for life
Access syntax	*p , p->m	as the variable itself

Assigning to a reference assigns to the object it refers to; assigning to a pointer makes it point somewhere else.

Passing Objects: Passing by value copies the whole object, which is expensive for large types such as a vector of prices. Pass by reference (or pointer) to avoid the copy, and by **const** reference when the function should only read.

9.3 const Correctness

const Reference Parameters: `double mean(const std::vector<double>& x)` reads without copying and without permission to modify. The guarantee is enforced by the compiler but can be defeated by an explicit cast, which should never be done.

const and Pointers: Read the declaration from right to left.

```
double a = 1.0, b = 2.0;
double* p1 = &a;           // change *p1 and p1
const double* p2 = &a;      // *p2 read only, p2 can move
double* const p3 = &a;      // *p3 writable, p3 fixed
const double* const p4 = &a; // neither can change
```

const Member Functions: A trailing **const** promises not to modify the object. Only such functions may be called on a **const** object, or through a **const** pointer or reference.

```
class Portfolio
{
    double m_value = 0.0;
public:
    void revalue(double v) { m_value = v; }
    double value() const { return m_value; }    // cannot assign m_value
};

const Portfolio& view = book;
view.value();    // fine
view.revalue(1.0);    // compile error
```

Access control is per class, not per object: a member function can read and write the private members of any other object of the same class passed to it.

9.4 Static Member Functions

A **static** member function belongs to the class rather than to an instance. It has no **this** pointer, is called as `ClassName::function(...)`, and is useful for factory functions or utilities that combine several objects of the class.

```
class Bond
{
public:
    static Bond par_bond(double face, double coupon, int years);
};
Bond b = Bond::par_bond(100.0, 0.04, 5);    // no object needed
```

10 Resource Management

10.1 Objects on the Heap

```
void run()
{
    Bond local(100.0, 0.05, 3);           // stack, freed automatically
    Bond* heap = new Bond(100.0, 0.05, 3); // heap
}    // heap object is never freed: leak

Bond* a = new Bond(100.0, 0.05, 3);
Bond* alias = a;
delete a;           // object gone, but a and alias still hold the old address
a = nullptr;       // dangling pointers are not reset automatically
alias = nullptr;
```

Arrays of Objects: `new Bond[n]` requires a default constructor and must be released with `delete[]`. Using plain `delete` on an array is undefined behaviour that compilers often fail to flag.

10.2 RAII

Resource Acquisition Is Initialisation: tie every resource (heap memory, file handle, lock) to the lifetime of an object. The constructor acquires it and the destructor releases it, so cleanup happens automatically when the object leaves scope, even if an exception is thrown. `std::string`, `std::vector` and `std::fstream` all follow this pattern.

10.3 The Rule of Three

If a class needs a custom **destructor**, it almost certainly also needs a custom **copy constructor** and **copy assignment operator**. The compiler generated versions copy the raw pointer, so two objects end up owning the same memory and both try to free it.

```
class PriceSeries
{
    double* m_data;
    std::size_t m_size;
public:
    explicit PriceSeries(std::size_t n)
        : m_data(new double[n]()), m_size(n) {}

    ~PriceSeries() { delete[] m_data; }

    // copy constructor: build a new, independent buffer
    PriceSeries(const PriceSeries& other)
        : m_data(new double[other.m_size]), m_size(other.m_size)
    {
        for (std::size_t i = 0; i < m_size; ++i)
            m_data[i] = other.m_data[i];
    }

    // copy assignment: replace existing contents
    PriceSeries& operator=(const PriceSeries& other)
    {
        if (this == &other)
            return *this;           // 1. self assignment
        if (m_size != other.m_size) // 2. reuse memory if possible
        {
            delete[] m_data;        // 3. release old resource
            m_data = nullptr;
            m_data = new double[other.m_size];
            m_size = other.m_size;
        }
        for (std::size_t i = 0; i < m_size; ++i)
            m_data[i] = other.m_data[i];    // 4. copy
```

```
        return *this;           // allows a = b = c
    }

    double& operator[](std::size_t i) { return m_data[i]; }
    double operator[](std::size_t i) const { return m_data[i]; }
    std::size_t size() const { return m_size; }
};
```

When Each Is Called: `PriceSeries b(a);` and `PriceSeries b = a;` create a new object and call the **copy constructor**. `b = a;` on an object that already exists calls **copy assignment**.

this: Inside a member function, **this** points to the current object; **this->m_size** and **m_size** are the same. Comparing **this == &other** detects self assignment.

10.4 Operator Overloading

Operators can be defined for user types so that code reads naturally. Returning a new object by value from `+` leaves both operands unchanged.

```
// element wise sum of two series of equal length
PriceSeries operator+(const PriceSeries& lhs, const PriceSeries& rhs)
{
    PriceSeries out(lhs.size());
    for (std::size_t i = 0; i < lhs.size(); ++i)
        out[i] = lhs[i] + rhs[i];
    return out;
}
```

The `const` overload of `operator[]` is what allows reading from the `const` operands.

10.5 Letting the Library Do It

std::vector: Already implements allocation, release and deep copying correctly. A class holding a `std::vector<double>` instead of a raw pointer needs none of the three special functions (the **rule of zero**).

Smart Pointers (<memory>): wrap a heap object and delete it automatically.

```
auto opt = std::make_unique<EuropeanOption>(100.0, 105.0, 0.03, 0.2, 1.0);
double v = opt->price();    // used like a raw pointer
auto owner = std::move(opt); // ownership transferred; opt is now null
```

`std::unique_ptr` has a single owner: it cannot be copied, only moved. `std::shared_ptr` allows shared ownership via reference counting when that is genuinely needed.

11 Templates

11.1 Function Templates

A template is a blueprint from which the compiler generates a separate function for every type it is used with. Several template parameters are allowed.

```
template <typename T>
T clamp_value(T x, T lo, T hi)
{
    return x < lo ? lo : (x > hi ? hi : x);
}

clamp_value(1.3, 0.0, 1.0); // T = double, returns 1.0
clamp_value(-4, 0, 10);    // T = int, returns 0
```

11.2 Class Templates

The types of members, and even compile time constants, become parameters. A concrete class is generated only when the template is instantiated. Non type parameters may be integral constants such as `int` or `std::size_t`.

```
template <typename T, std::size_t N>
class RollingWindow
{
    std::array<T, N> m_buf{};
    std::size_t m_count = 0;
public:
    void push(T x) { m_buf[m_count++ % N] = x; }
    T mean() const
    {
        std::size_t n = m_count < N ? m_count : N;
        T sum = T();
        for (std::size_t i = 0; i < n; ++i)
            sum += m_buf[i];
        return sum / static_cast<T>(n);
    }
};
```

```
| RollingWindow<double, 20> ma20; // 20 period moving average
```

Any type used for `T` must support every operation the template uses (here `+=`, `/` and a default value).

11.3 Variadic Templates (C++11)

A **parameter pack** (`typename... Ts`) accepts any number of template arguments. Packs are usually processed recursively, with a single argument overload as the base case, and combined with `constexpr` they allow results to be computed at compile time.

```
// base case: one cash flow at time t
template <typename T>
constexpr double pv(double df, int t, T cf)
{
    double d = 1.0;
    for (int i = 0; i < t; ++i) d *= df;
    return cf * d;
}
// recursive case: first cash flow plus the rest one period later
template <typename T, typename... Rest>
constexpr double pv(double df, int t, T cf, Rest... rest)
{
    return pv(df, t, cf) + pv(df, t + 1, rest...);
}
pv(0.95, 1, 5.0, 5.0, 105.0); // three year 5% bond, flat discount factor
```

11.4 Practical Guidance

- Templates move work from runtime to compile time: faster, more flexible code at the price of longer builds, especially with deep recursion
- Compiler errors can be long; test with a few concrete types first
- Put template definitions in headers, since the compiler needs the full body wherever it instantiates the template

Spectrum of Abstraction:

Construct	Types	Implementation
Concrete class	Fixed	Complete
Abstract class	Fixed	Partial
Interface	Fixed	None
Class template	Generic	Partial or complete
Template interface	Generic	None

Runtime abstraction (virtual functions) buys flexibility with some runtime cost. Compile time abstraction (templates) buys even more flexibility with no dispatch cost, but produces larger executables and is harder to debug.

12 Namespaces

12.1 Purpose

Large codebases, or code combined from several libraries, inevitably reuse common names such as `Curve` or `Model`. Two global classes with the same name cause a redefinition error. A **namespace** gives each set of names its own qualified prefix, so `pricing::Model` and `risk::Model` coexist.

12.2 Declaring and Extending

```
// curves.hpp
namespace pricing
{
    constexpr double bp = 1e-4;
    class Curve
    {
    public:
        double rate(double t) const;
    };
}

// curves.cpp
double pricing::Curve::rate(double t) const // qualified definition
{
```

```
|     return 0.03 + 10 * bp * t;
| }

```

Namespaces are **open**: the same namespace can be reopened in any number of files, each adding more declarations.

12.3 Referring to Names

- **Qualified name:** `pricing::Curve c`; always works and is unambiguous
- **Using declaration:** `using pricing::Curve`; imports one name into the current scope
- **Using directive:** `using namespace pricing`; imports every name; avoid it in headers, where it leaks into every file that includes them

A qualified name always wins: after `using namespace risk`, `Model` refers to `risk::Model` but `pricing::Model` is still reachable.

Nesting and Aliases:

```
namespace pricing
{
    namespace equity { class Model {}; }
    namespace rates { class Model {}; }
}
namespace eq = pricing::equity; // alias
eq::Model m;                  // pricing::equity::Model

```

Since C++17 the nested form can be written `namespace pricing::equity { ... }`.

13 Designing a Class Hierarchy

13.1 Kinds of Class

Kind	Characteristic
Concrete class	Every member function has an implementation
Abstract class	At least one pure virtual function (= 0)
Interface	Only pure virtual functions, no data

Abstract classes and interfaces cannot be instantiated; they define a contract that derived classes fulfil.

13.2 Example: A Simple Market Simulator

A time stepping simulation illustrates how responsibilities can be divided.

- **Clock** interface (`TimeStepped`): anything with an `advance(dt)` method
- **Asset**: holds its current price and evolves it each step (e.g. by geometric Brownian motion)
- **Market**: owns a collection of assets and advances them together
- **Observer** interface: notified after each step; a **PriceRecorder** keeps the last *n* prices of an asset for later analysis
- **Agent** abstract class: reacts to the market each step; a **Trader** holds a **Portfolio** (a map from ticker to quantity) and issues buy or sell orders

Extension: Hedging Desk. A **Dealer** agent writes options (derived from an abstract `Option` with `Call` and `Put` subclasses), charges the risk neutral price, and rebalances a hedge portfolio each step. A **SettlementObserver** records the underlying price at each option’s expiry so the final profit and loss of the hedge can be measured.

Design Guidelines:

- Give each class one clear responsibility
- Depend on interfaces rather than concrete classes, so components can be swapped (e.g. a different price model)
- Keep ownership explicit, preferably with `std::unique_ptr` or containers of values

14 Error Handling

14.1 Assertions

`assert(expr)` from `<cassert>` aborts the program with a diagnostic if **expr** is false. Assertions document assumptions that should be impossible to violate if the code is correct. Defining `NDEBUG` before the include (as release builds typically do) removes them entirely.

```
double forward(double spot, double r, double t)
{
    assert(t >= 0.0);    // negative time is a programming error
    return spot * std::exp(r * t);
}

```

14.2 Exceptions

An exception signals that a function cannot fulfil its contract, typically because of something outside the programmer’s control: invalid input data, a missing file, a failed allocation or an unreachable data feed. The function **throws**; control unwinds up the call stack to the nearest matching handler.

Syntax: `throw` raises, `try` marks code that may raise, and `catch` handles.

```
double implied_vol(double price, double spot, double strike)
{
    if (price < std::max(spot - strike, 0.0))
        throw std::invalid_argument("price below intrinsic value");
    /* root finding ... */
    return 0.2;
}

try
{
    double iv = implied_vol(1.0, 110.0, 100.0);
}
catch (const std::invalid_argument& e)
{
    std::cerr << "Skipping quote: " << e.what() << "\n";
}

```

Matching Handlers: Catch blocks are tried in order, and the first whose type matches is used. No numeric conversions are applied (a thrown `int` is not caught by `catch (double)`), although a derived exception class is caught by a handler for its base. `catch (...)` catches anything and belongs last.

```
try { load_market_data("quotes.csv"); }
catch (const std::invalid_argument& e) { /* bad value */ }
catch (const std::exception& e) { /* any standard error */ }
catch (...) { /* anything else */ }

```

Exception Types: Throw objects rather than raw integers or strings. Standard classes such as `std::runtime_error` carry a message, and a custom class derived from `std::exception` can carry context or a recovery method. Catch by `const` reference.

Where to Put try: Validate at the boundary where untrusted data enters (user input, files, network), handle the error there, and let internal routines assume valid data, guarded by assertions.

14.3 Choosing a Mechanism

- **Ordinary branching** (or returning a status value) for outcomes that are part of normal operation
- **Exceptions** for genuine failures beyond the program’s control; avoid them for routine control flow in performance critical loops
- **Assertions** for conditions that indicate a bug if they ever fail

Functions that guarantee not to throw can be marked **noexcept**, which also enables some compiler optimisations.